PROJETO Próxima Etapa

Testes e Qualidade de Software (DevOps)

INTEGRANTES DO PROJETO e RA'S

Arthur Carvalho Chagas	-	25028070
Cauã Aono	-	25028199
Lucas Pereira	-	23011167

São Paulo

2026

Sumário

1.1. Apresentar 4 testes unitários 3
1.2. Apresentar 2 testes de integração 7
1.3. Relatório de teste de carga do servidor 10
1.4. Apresentar um teste de usuário (SISTEMA) 12
1.5. Critérios de aceitação 14

INTRODUÇÃO

O projeto Próxima Etapa – App do Aluno é um aplicativo Android feito em Java para os estudantes da ONG Próxima Etapa. Pelo app o aluno faz login, consulta os cursos e a sua agenda, registra presença por QR Code, acompanha o progresso, vê o card digital e os certificados, faz o teste de perfil e conversa com a equipe da ONG. Os dados ficam salvos no próprio celular em um banco SQLite e são sincronizados com uma API em PHP, que guarda as informações em um banco MySQL rodando no XAMPP. Como o check-in precisa funcionar mesmo sem internet, a presença é gravada primeiro no aparelho e enviada para a API na próxima sincronização, e boa parte dos testes foi pensada em cima desse comportamento.

Os testes unitários foram escritos com JUnit 4 e ficam na pasta app/src/test do projeto. Os testes de integração e de sistema foram feitos no emulador do Android Studio (Pixel 8, API 35), com a API rodando no XAMPP e os dados acompanhados pelo phpMyAdmin.

1. Teste Unitário De Software

- Função: A função getPercentual() é responsável por calcular quanto do curso o aluno já concluiu, dividindo os encontros com presença pelo total de encontros. O valor aparece na barra de progresso e define quando o certificado é liberado.

```
public int getPercentual() {
    if (totalAulas <= 0) {
        return 0;
    }
    return Math.min(100, aulasAssistidas * 100 / totalAulas);
}
```

Caso	Entrada	Resultado esperado	Status
1. Nenhuma aula	0 aulas assistidas de 4	Retornar 0.	Aprovado
2. Metade do curso	2 de 4	Retornar 50.	Aprovado
3. Divisão não exata	1 de 3	Retornar 33, arredondando para baixo.	Aprovado
4. Curso completo	4 de 4	Retornar 100.	Aprovado
5. Mais aulas que o total	5 de 4	Limitar o valor em 100.	Aprovado

Caso	Entrada	Resultado esperado	Status
6. Total igual a zero	3 de 0	Retornar 0 sem erro de divisão por zero.	Aprovado

O caso 6 foi colocado porque, se algum curso fosse cadastrado sem total de aulas, a conta aulasAssistidas * 100 / totalAulas daria divisão por zero e o aplicativo fecharia. O if no começo da função evita esse problema.

- Função: A função isDiaUnico() é responsável por verificar se o curso começa e termina na mesma data, para que os cursos de um dia só apareçam com a faixa amarela e a etiqueta DIA ÚNICO na lista de cursos e na agenda.

```
public boolean isDiaUnico() {
    return dataInicio != null && dataInicio.equals(dataFim);
}
```

Caso	Entrada	Resultado esperado	Status
1. Mesma data	Início 2026-11-21; Fim 2026-11-21	Retornar true.	Aprovado
2. Datas diferentes	Início 2026-11-10; Fim 2026-12-01	Retornar false.	Aprovado
3. Dias seguidos	Início 2026-11-21; Fim 2026-11-22	Retornar false.	Aprovado
4. Sem data de início	Início null; Fim 2026-11-21	Retornar false sem erro.	Aprovado
5. Sem data de fim	Início 2026-11-21; Fim null	Retornar false.	Aprovado

Nos casos 4 e 5 a ideia foi ver se a função quebra quando a API manda um curso sem data. Como a comparação começa por dataInicio != null, o retorno é false e o curso aparece como um curso normal.

- Função: A função formatarData() é responsável por converter as datas que vêm da API e do banco (2026-11-12) para o formato mostrado ao aluno (12/11/2026), usado na agenda, no histórico de presenças e nos certificados. A formatarDataHora() usa a mesma conversão e acrescenta o horário.

```
private static Date lerData(String isoData) {
    if (isoData == null || isoData.length() < 10) {
        return null;
    }
    try {
        return new SimpleDateFormat("yyyy-MM-dd", Locale.ROOT)
                .parse(isoData.substring(0, 10));
    } catch (ParseException e) {
        return null;
    }
}

public static String formatarData(String isoData) {
    Date data = lerData(isoData);
    return data == null ? "" : new SimpleDateFormat("dd/MM/yyyy",
PT_BR).format(data);
}
```

Caso	Entrada	Resultado esperado	Status
1. Data válida	"2026-11-12"	Retornar "12/11/2026".	Aprovado
2. Data com hora	"2026-11-08 13:05:00"	Retornar "08/11/2026", ignorando a hora.	Aprovado
3. Data e hora do check-in	"2026-11-08 13:05:00" na formatarDataHora()	Retornar "08/11/2026 às 13:05".	Aprovado
4. Texto vazio	""	Retornar texto vazio, sem erro.	Aprovado
5. Valor nulo	null	Retornar texto vazio, sem erro.	Aprovado
6. Texto que não é data	"data-errada"	Retornar texto vazio, sem erro.	Aprovado

Os casos 4, 5 e 6 mostram que, quando a data vem vazia ou errada, a tela fica sem a data em vez de o aplicativo parar. Isso acontece porque a lerData() devolve null e a formatarData() trata esse null.

- Função: A função nome() é responsável por transformar o código da área salvo depois do teste de perfil (ADM, FIN, COM ou TEC) no nome mostrado na tela de resultado e na aba Perfil, tratando também o aluno que ainda não fez o teste.

```
public static String nome(String area) {
    if (ADM.equals(area)) return "Administração e Gestão";
    if (FIN.equals(area)) return "Contábil e Financeira";
    if (COM.equals(area)) return "Comunicação e Publicidade";
    if (TEC.equals(area)) return "Tecnologia e Inovação";
    return "Ainda não definido";
}
```

Caso	Entrada	Resultado esperado	Status
1. Administração	"ADM"	Retornar "Administração e Gestão".	Aprovado
2. Financeira	"FIN"	Retornar "Contábil e Financeira".	Aprovado
3. Comunicação	"COM"	Retornar "Comunicação e Publicidade".	Aprovado
4. Tecnologia	"TEC"	Retornar "Tecnologia e Inovação".	Aprovado
5. Aluno sem teste	"" (vazio)	Retornar "Ainda não definido".	Aprovado
6. Código em minúsculo	"tec"	Retornar "Ainda não definido", pois a comparação diferencia maiúsculas.	Aprovado

Os quatro testes foram executados no Android Studio (botão direito na pasta test > Run Tests) e os 23 casos passaram.

2. Teste De Integração De Software

- As funções que irão ser analisadas em conjunto são registrar() e registrarAula().

Função registrar(): Responsável por receber o código lido do QR Code, procurar o encontro na agenda do aluno, conferir se a presença ainda não foi registrada e gravar o check-in no banco local como pendente de envio.

Função registrarAula(): Responsável por somar uma aula no progresso da inscrição do aluno e, quando ele chega no total de encontros, marcar o curso como concluído e gerar o código do certificado.

```
ItemAgenda encontro = new AgendaDao(context).buscarPorCodigo(codigo);
if (encontro == null) {
    return new Resultado(false, "QR Code inválido", ...);
}
CheckinDao checkinDao = new CheckinDao(context);
if (checkinDao.existeParaAgenda(encontro.getId())) {
    return new Resultado(false, "Presença já registrada", ...);
}
checkinDao.inserir(encontro.getId(), codigo, DateUtils.agora());
boolean concluiuCurso = new
InscricaoDao(context).registrarAula(encontro.getCursoId());
```

Funcionamento em conjunto: As duas funções trabalham juntas para que a leitura do QR Code atualize todo o acompanhamento do aluno. A registrar() só grava a presença se o código pertencer a um encontro da agenda e se ainda não houver check-in nele. Depois de gravar, ela chama a registrarAula() com o curso do encontro, e se o curso for concluído nesse momento o app cria uma notificação avisando que o certificado está disponível.

Inicialmente, foi verificado se a função registrar() encontrou o encontro a partir do código digitado na aba Check-in, usando a conta de demonstração e os códigos que já estão no banco. Em seguida, foi conferido se a presença apareceu no histórico como pendente e se a registrarAula() atualizou o progresso na tela Minhas inscrições. Alguns códigos também foram lidos pela câmera, usando a página qrcodes_teste.html.

Também foram avaliadas situações de erro, como um código que não existe, um encontro que já tinha presença e um código digitado com espaços. Dessa forma, foi possível analisar se as duas funções mantêm a agenda, o histórico e o progresso do curso coerentes entre si.

Cenários colocados e resultado esperado:

- Ler o código PE-ENC-004 (Simulação da ONU, curso de dia único com 1 encontro). / A presença deverá ser registrada, o curso concluído com 100% e o certificado liberado.
- Ler o código PE-ENC-009 (Educação Financeira, 3 de 6 encontros feitos). / A presença deverá ser registrada e o progresso ir de 50% para 66%, sem gerar certificado.
- Ler de novo um código que já tem presença (PE-ENC-001). / O sistema deverá avisar que a presença já foi registrada e não duplicar o check-in.
- Digitar um código que não existe (PE-ENC-999). / O sistema deverá informar QR Code inválido e não gravar nada.
- Digitar o código com espaços antes e depois. / O sistema deverá remover os espaços e registrar normalmente.

Cenário	Resultado previsto	Status
Ler o código PE-ENC-004.	Presença registrada, curso concluído e certificado disponível.	Aprovado
Ler o código PE-ENC-009.	Presença registrada e progresso de 50% para 66%.	Aprovado
Ler um código que já tem presença.	Mensagem “Presença já registrada” e nenhum check-in duplicado.	Aprovado
Digitar um código inexistente.	Mensagem “QR Code inválido” e nada gravado no banco local.	Aprovado
Digitar o código com espaços.	Espaços removidos e presença registrada.	Aprovado

- As próximas a serem utilizadas serão listarPendentes() e sincronizar().

Função listarPendentes(): Responsável por buscar no banco do celular os check-ins que ainda não foram enviados para a API, ou seja, os que estão marcados como PENDENTE.

Função sincronizar(): Responsável por enviar os pendentes para a API (script checkins.php), marcar cada um como enviado e depois baixar os dados atualizados do servidor. Se a API não responder, ela mantém os dados salvos no aparelho e avisa o aluno.

```
CheckinDao checkinDao = new CheckinDao(context);
for (Checkin checkin : checkinDao.listarPendentes()) {
    int remoteId = api.enviarCheckin(alunoId, checkin);
    checkinDao.marcarSincronizado(checkin.getId(), remoteId);
    resultado.enviados++;
}
...
} catch (IOException e) {
    // API fora do ar: o app continua com os dados do SQLite
    sessao.setModoOffline(true);
    resultado.mensagem = "Não foi possível acessar a API. Mostrando os
dados salvos no aparelho.";
}
```

Funcionamento em conjunto: O check-in é sempre salvo primeiro no celular. A sincronizar() pede para a listarPendentes() o que ainda não foi enviado e manda cada registro por POST. No servidor, a API confere se o QR Code pertence a um encontro daquele aluno e a chave única (aluno_id, agenda_id) impede presença duplicada. Quando a API confirma, o registro passa de PENDENTE para ENVIADO.

O teste de integração teve como objetivo verificar a comunicação entre o aplicativo e o servidor, principalmente quando a conexão cai. Para simular a queda do servidor, o Apache foi parado no XAMPP Control Panel, e para simular o celular sem internet foi ativado o modo avião no emulador.

Em cada cenário foi conferido o status do check-in no Histórico do aplicativo e a tabela checkins no phpMyAdmin, para ver se o que estava no celular batia com o que chegou no banco MySQL. Também foi verificado se a data e hora gravadas no servidor eram as do momento da leitura, e não as do envio.

Cenários colocados e resultados esperados:

- Fazer o check-in com o Apache ligado. | A presença deverá ser enviada na hora e aparecer na tabela checkins.
- Parar o Apache e fazer o check-in PE-ENC-009. | A presença deverá ficar salva no celular como PENDENTE e o app avisar que a API não pôde ser acessada.
- Ligar o Apache de novo e sincronizar pelo Histórico. | O status deverá mudar para ENVIADO e o registro aparecer no MySQL com a data e hora da leitura.
- Fazer o check-in com o emulador em modo avião. | O sistema deverá avisar que está sem conexão e manter a presença pendente.
- Sincronizar duas vezes seguidas. | Nada deverá ser reenviado e a tabela não deverá ter linha duplicada.

Cenário de teste	Resultado obtido	Status
Check-in com o Apache ligado.	Presença enviada, status ENVIADO e registro novo na tabela checkins.	Aprovado
Check-in com o Apache parado.	Presença salva como PENDENTE e aviso de que a API não pôde ser acessada.	Aprovado
Sincronizar depois de ligar o Apache.	Status ENVIADO e registro no MySQL com a data e hora da leitura.	Aprovado
Check-in em modo avião.	Mensagem “Sem conexão. Seus dados ficaram salvos…” e presença pendente.	Aprovado
Sincronizar duas vezes seguidas.	Nada reenviado e nenhuma linha duplicada na tabela.	Aprovado

Os testes de integração tiveram como objetivo verificar o funcionamento conjunto das funções registrar(), registrarAula(), listarPendentes() e sincronizar(). Foram avaliados o registro de presença, a atualização do progresso, o envio para a API e o comportamento sem conexão. O cenário mais importante para a ONG foi o do servidor desligado, porque muitos encontros acontecem em lugares com internet ruim, e nele a presença ficou guardada e foi enviada assim que o servidor voltou.

Relatório de Teste de Carga do Servidor

O que irá ser analisado: O teste de carga teve como objetivo medir como a API em PHP e o banco MySQL se comportam quando vários alunos usam o aplicativo ao mesmo tempo, como no começo de um encontro, quando a turma inteira faz o check-in e sincroniza quase junto. A ideia foi descobrir quantas requisições por segundo o servidor aguenta, quanto tempo ele leva para responder e se aparecem erros conforme o número de usuários aumenta.

Ferramenta utilizada: Para gerar a carga foi usado o Apache Bench (ab), que já vem instalado junto com o XAMPP na pasta C:\xampp\apache\bin. Ele dispara várias requisições ao mesmo tempo para o endereço informado e, no final, mostra um relatório com o tempo de resposta e a quantidade de falhas. O teste foi feito no mesmo computador do XAMPP, usando o endereço localhost no lugar do 10.0.2.2 que o emulador usa, com o Apache e o MySQL ligados e o banco proxima_etapa importado pelo arquivo banco.sql.

Comandos executados no Prompt de Comando:

```
cd C:\xampp\apache\bin
ab -n 1000 -c 10  http://localhost/proxima_etapa_api/checkins.php
ab -n 1000 -c 50  http://localhost/proxima_etapa_api/checkins.php
ab -n 1000 -c 100 http://localhost/proxima_etapa_api/checkins.php
```

Cenários colocados (o -n é o total de requisições e o -c é quantas são enviadas ao mesmo tempo, ou seja, quantos alunos usam o app juntos):

Cenário	Requisições (-n)	Simultâneas (-c)	Situação simulada
1	1000	10	Uma turma sincronizando no começo do encontro.
2	1000	50	Várias turmas usando o app no mesmo horário.
3	1000	100	Pico de uso, acima do esperado para a ONG.

Critérios definidos para considerar o servidor aprovado:

- Nenhuma requisição com falha (Failed requests igual a 0) nos cenários 1 e 2.
- Tempo médio de resposta abaixo de 1 segundo (1000 ms) com 50 usuários simultâneos.
- Apache e MySQL continuarem respondendo depois do teste, sem precisar reiniciar o XAMPP.

Os valores abaixo foram tirados do relatório do ab: Requests per second, Time per request (mean), a linha 95% da tabela de percentuais e Failed requests.

Cenário	Req./s	Tempo médio (ms)	95% até (ms)	Falhas	Status
1 (10 simultâneas)	_____	_____	_____	_____	_____
2 (50 simultâneas)	_____	_____	_____	_____	_____
3 (100 simultâneas)	_____	_____	_____	_____	_____

Com os resultados obtidos, o servidor respondeu em média em _____ ms com 10 alunos e em _____ ms com 50 alunos ao mesmo tempo, com _____ requisições com falha. No cenário de pico, com 100 alunos, o tempo médio foi de _____ ms. Esse teste é importante para a ONG porque o check-in costuma acontecer com a turma toda junta no início do encontro, e é nesse momento que a API recebe mais requisições de uma só vez.

3. Teste Sistema De Software

O que irá ser analisado: Os testes de sistema tiveram como objetivo avaliar o aplicativo Próxima Etapa inteiro, do jeito que um aluno usaria, do login até o certificado, com o app ligado à API e ao banco MySQL. A ideia foi verificar se as telas conversam entre si, se a informação que entra em uma parte aparece certa nas outras e se o app se comporta bem quando perde a conexão. Como teste de aceitação, cada funcionalidade foi comparada com o que foi pedido para o projeto: o aluno precisa conseguir fazer sozinho o que hoje depende da equipe da ONG.

No sistema Próxima Etapa, o teste de sistema consiste em usar o aplicativo completo simulando um aluno, desde o momento em que ele entra no app até a hora em que recebe o certificado de um curso concluído. Os testes foram feitos com a conta de demonstração, que já tem cursos em andamento, cursos concluídos e encontros com e sem presença, o que permitiu passar por todas as telas sem precisar cadastrar dados novos.

Ambiente utilizado nos testes:

Item	Configuração
Aparelho	Emulador do Android Studio, Pixel 8 com API 35 (Android 15)
Servidor	XAMPP com Apache e MySQL rodando no mesmo computador
Endereço da API no app	http://10.0.2.2/proxima_etapa_api/
Banco de dados	proxima_etapa, importado pelo arquivo banco.sql
Conta de teste	aluno@proximaetapa.org (senha 123456)
Acompanhamento	phpMyAdmin, para conferir as tabelas checkins, inscricoes e mensagens

A análise foi organizada da seguinte maneira:

1. Acesso ao aplicativo: login com a conta de demonstração, tentativa com senha errada e saída pelo logout.
2. Cursos: lista com os filtros Todos, Meus cursos e Dia único, abertura dos detalhes e inscrição em um curso.
3. Agenda: conferência dos encontros do aluno e do destaque para os cursos de dia único.
4. Presença: check-in pela câmera e pelo código digitado, com o servidor ligado e desligado.

5. Histórico: status ENVIADO e PENDENTE e sincronização manual.

6. Card digital e certificados: cursos concluídos no card, QR Code no verso e compartilhamento do certificado.

7. Teste de perfil: respostas, resultado e cursos recomendados.

8. Notificações e chat: uma notificação e uma mensagem foram inseridas pelo phpMyAdmin e conferidas no app depois da sincronização.

9. Perfil: edição dos dados, troca da foto e mudança do endereço da API.

O que foi verificado no código: o projeto possui uma Activity ou Fragment para cada funcionalidade, classes DAO para o banco SQLite e as classes ApiService e SyncManager para a comunicação com a API.

Funcionalidade	Resultado esperado	Status
Login e logout	Acesso com dados corretos, erro com senha errada e sessão mantida ao reabrir.	Aprovado
Cursos	Filtros funcionando e inscrição aparecendo em Minhas inscrições.	Aprovado
Curso de dia único	Faixa amarela e etiqueta DIA ÚNICO na lista e na agenda.	Aprovado
Agenda individual	Somente os encontros do aluno, com presença confirmada em verde.	Aprovado
Presença por QR Code	Registro pela câmera e pelo código, inclusive sem internet.	Aprovado
Sincronização	Pendentes enviados quando a API volta e “Sincronizado em…” no topo.	Aprovado
Histórico	Presenças listadas com o status certo.	Aprovado
Notificações	Aviso na área de notificações, no contador do sino e na barra do celular.	Aprovado
Card digital	Dados do aluno, cursos concluídos e QR Code de identificação.	Aprovado
Certificados	Somente cursos com 100% de presença, com opção de compartilhar.	Aprovado
Teste de perfil	Todas as respostas exigidas, área exibida e cursos recomendados.	Aprovado
Chat com a ONG	Mensagem gravada na tabela mensagens e resposta da ONG exibida no app.	Aprovado

Funcionalidade	Resultado esperado	Status
Perfil do aluno	Alterações salvas no aparelho e enviadas para a API.	Aprovado
Modo offline	App aberto com os dados salvos e aviso de modo offline.	Aprovado

Concluiu-se que os testes de sistema permitiram avaliar o aplicativo Próxima Etapa como um todo, do ponto de vista do aluno. Todas as funcionalidades atenderam aos resultados esperados, inclusive sem conexão com a API. Durante os testes foram observados dois pontos de configuração, a demora do Gradle na primeira abertura do projeto e a troca do endereço da API no celular físico, que foram explicados no guia de execução que acompanha o projeto. Dessa forma, considera-se que o aplicativo está pronto para ser usado pelos alunos da ONG.

Critérios de aceitação: Para fechar o teste de aceitação, cada requisito do aplicativo foi relacionado com a funcionalidade testada na tabela anterior. O requisito foi considerado atendido quando o aluno conseguiu fazer a tarefa sozinho, sem depender da equipe da ONG.

Nº	Requisito do cliente / usuário	Funcionalidade testada	Situação
R1	O aluno entrar no app com o próprio login.	Login e logout	Atendido
R2	Consultar os cursos e se inscrever.	Cursos; Curso de dia único	Atendido
R3	Ver a própria agenda de encontros.	Agenda individual	Atendido
R4	Registrar presença por QR Code, mesmo offline.	Presença por QR; Modo offline	Atendido
R5	Presenças enviadas à ONG sem duplicar.	Sincronização; Histórico	Atendido
R6	Acompanhar o progresso e obter o certificado.	Certificados; Notificações	Atendido
R7	Ter um card digital de identificação.	Card digital	Atendido
R8	Descobrir a área profissional mais indicada.	Teste de perfil	Atendido
R9	Falar com a equipe da ONG pelo app.	Chat com a ONG	Atendido
R10	Manter os próprios dados atualizados.	Perfil do aluno	Atendido

Como os dez requisitos foram atendidos, o teste de aceitação confirma que o aplicativo faz o que foi pedido pela ONG e pode ser colocado em uso pelos alunos.